

Classify & Learn Lab

Machine Learning Desktop Application for Vehicle Analytics

Final Project Report

Prepared by: Team -1

Business Intelligence Project

August 2026

Contents

1. Executive Summary	3
2. Project Objectives	3
3. Technology Stack	3
4. System Architecture	4
5. Application Walkthrough	4
5.1 Data & Features.....	4
5.2 Models	5
5.3 Results	5
5.4 Playback and Annotation	5
6. Project Data Model.....	6
7. Development Process	6
7.1 Automated Testing	6
8. Challenges and Resolutions	6
9. Application overview	7
10. Conclusion	11
10.1 Future Work.....	11

1. Executive Summary

Classify & Learn Lab is a desktop machine learning application built with Python and PySide6 that lets a user take a raw vehicle-telemetry dataset (steering angle, throttle, brake pressure, gaze position, heart rate, and similar driving/simulator signals) through a complete analytics workflow inside a single guided interface: importing and cleaning data, engineering and selecting features, visualizing patterns, training and evaluating machine learning models, and reviewing results alongside synchronized video playback of the recorded session.

The application is organized into four top-level workflow areas — Data & Features, Models, Results, and Playback & Annotation — and is designed to be usable by someone without a coding background, while still exposing the underlying statistical and machine-learning configuration (feature selection method, model type, hyperparameters, missing-value handling) for more advanced use.

2. Project Objectives

- Provide a single desktop tool that covers the full analytics pipeline for vehicle telemetry data, from raw CSV to a trained, testable classifier.
- Make data cleaning and feature engineering approachable through a point-and-click interface, without requiring the user to write code.
- Support multiple feature-selection and dimensionality-reduction techniques (correlation, mutual information, mRMR, PCA) so users can compare approaches.
- Support supervised, semi-supervised, and unsupervised machine learning workflows within one consistent application shell.
- Allow recorded driving sessions to be reviewed as synchronized video and sensor-signal playback, with annotation support.
- Keep the codebase modular (data, feature, preprocessing, analysis, model, frontend layers) so the ML backend and the PySide6 GUI can evolve independently.

3. Technology Stack

Layer	Technology
Language	Python 3.12+
Desktop UI Framework	PySide6 (Qt for Python)
Data Handling	pandas, numpy
Machine Learning	scikit-learn (classification, clustering, semi-supervised self-training)
Feature Selection	mrmr-selection, mutual information (scikit-learn)
Charting / Visualization	matplotlib, embedded via FigureCanvasQTAgg
Packaging	PyInstaller (--onedir --windowed build)

Layer	Technology
Testing	pytest
Persistence	joblib (model export), JSON metadata, .icp project files
Video / Streaming	UDP streaming for live sensor playback synced to video

4. System Architecture

The codebase follows a layered structure that separates the reusable analytics/ML backend from the PySide6 presentation layer, so the same underlying functions can be called from the GUI, from the pytest test suite, or from a plain command-line script.

Package	Responsibility
src/data	Loading datasets, saving/loading .icp project files
src/preprocessing	Missing-value imputation, standardization, normalization
src/feature	Statistical feature extraction and windowed feature datasets for time-series signals
src/analysis	PCA, correlation analysis, mutual information, and mRMR feature-selection analysis
src/evaluation	Shared classification and clustering metrics (accuracy, precision, recall, F1, silhouette, etc.)
src/model	Model factory, training routines, and supervised / semi-supervised / unsupervised workflows
src/frontend	PySide6 windows, tabs, charts, and widgets that present the backend to the user
src/playback_annotation	Video/sensor synchronization and UDP live-stream playback
src/tests	Automated tests covering the analytics and model backend

5. Application Walkthrough

5.1 Data & Features

- **Import:** upload a dataset (CSV or similar), preview it, review a missing-values summary, and select a label column for supervised tasks.
- **Feature Extraction:** compute statistical features per column and build windowed feature datasets from time-series signals.
- **Analysis:** run PCA, correlation analysis, mutual information, and mRMR feature ranking to understand and select the most relevant features.
- **Visualization:** render Histogram, Scatter, Line, Box Plot, Bar Chart, Grouped Box Plot, Class Separation, 3D Scatter, Time Series (All Signals), Feature Distribution Comparison, and Correlation Heatmap charts, with adjustable histogram bin counts and the option to export any rendered chart as a PNG, PDF, or SVG file.

5.2 Models

The Models tab lets a user train a classifier directly against the active dataset without leaving the GUI. A model-type dropdown (SVM, KNN, Decision Tree, Random Forest, Logistic Regression) drives a parameter panel that only shows the fields relevant to the selected algorithm, alongside test-split ratio, random-state seed, and a missing-value fill strategy (median, mean, mode, or none).

- **Train:** runs the full supervised workflow — data preparation, model construction, train/test split, fitting, and evaluation — and adds the trained model to the project.
- **Manage:** lists every saved model in the project with its algorithm, parameters, and evaluation metrics.
- **Delete:** removes a saved model from the project after confirmation.
- **Test:** evaluates one or more selected saved models against the current working dataset and displays accuracy, precision, recall, and F1 side by side.

The backend also supports semi-supervised learning (self-training on partially labeled data, tracking how many rows become pseudo-labeled over successive iterations) and unsupervised clustering (K-Means, DBSCAN, and Hierarchical clustering, each evaluated with silhouette, Davies-Bouldin, and Calinski-Harabasz scores, plus PCA-based 2D cluster visualization).

In its more complete form, the Models tab generalizes into a Model Builder: a Category dropdown (supervised / semi_supervised / unsupervised) drives the available algorithms, models can be added to a queue and trained together in a single batch, and both queued and previously trained models are listed with their category, algorithm, and training status.

5.3 Results

The Results tab lists every trained model — across all three learning categories — in one table, alongside its source, algorithm, and the relevant evaluation metrics (accuracy, precision, recall, and F1 for classifiers; silhouette, Davies-Bouldin, and Calinski-Harabasz scores for clustering models), with per-model training info and confusion-matrix panels below. A Comparisons sub-tab lets the user select several models and generate a combined metrics image and confusion-matrix image, exportable as a PNG.

5.4 Playback and Annotation

The Playback & Annotation area synchronizes recorded video with the corresponding sensor-signal stream (steering angle, throttle, brake pressure, gaze position, heart rate) so a reviewer can watch a driving session while the underlying telemetry plays back in step, supporting both recorded-file playback and live UDP-streamed data. It also provides manual annotation and label correction — loading a pretrained model to generate predictions per row, letting the user record a corrected label and note where the prediction is wrong, and exporting both the corrected dataset and a full annotation log.

6. Project Data Model

Each analysis session is saved as a project (.icp file) capturing the dataset reference, selected feature columns, label column, applied preprocessing steps, and any trained models — so a user can close the application and resume an in-progress project later. Trained models can additionally be exported as standalone .pkl files with accompanying JSON metadata (algorithm, parameters, metrics, feature columns) for use outside the application.

7. Development Process

The project was developed collaboratively with a team, using Git branches to isolate feature work (for example, a `model_feature` branch for the initial Models tab implementation, and a `model_feature_merge` branch reconciling it with a parallel backend refactor). The backend was refactored midway through development to separate model construction (`model_factory.py`), data preparation (`model_utils.py`), training (`model_training.py`), result formatting (`result_builders.py`), and metrics (`evaluation/metrics.py`) into distinct, independently testable modules, and the frontend Models tab was subsequently updated to call the new `run_supervised_workflow()` function rather than the earlier, now-removed `train_model()` implementation.

7.1 Automated Testing

The `src/tests` package includes automated pytest coverage for the analysis functions, the model factory, model training, model utilities, preprocessing, and the supervised, semi-supervised, and unsupervised workflows, alongside a dedicated test for the main application window.

8. Challenges and Resolutions

- **Missing values breaking model training:** algorithms such as KNN and SVM initially failed on datasets with missing cells. This was resolved by adding an automatic fill-method (median / mean / mode) directly into the data-preparation step used by every training and evaluation call.
- **Continuous vs. categorical labels:** a regression-style label (e.g., an admission-chance probability) is not directly usable by the classifiers the application supports; the workflow addressed this by bucketing the continuous value into discrete categories before training.
- **UI rendering glitches:** occasional overlapping form fields in the Models tab sidebar were traced to a Windows display-scaling repaint issue in Qt rather than a layout defect, and were resolved by forcing a window redraw.
- **Mid-project backend refactor:** a restructuring of the `src/model` package (introducing `model_factory.py`, `result_builders.py`, and workflow-specific modules) required the already-built Models tab to be re-wired to the new `run_supervised_workflow()` function and its nested metrics structure.

9. Application overview

The following screenshots show the running desktop application: the Import view for loading and previewing a dataset, the Visualization tab's extended chart-type selection, and the Models tab training and reviewing saved classifiers.

The screenshots below show the application in a later, more complete state, including the full Feature Extraction and Analysis workflows, an expanded Model Builder supporting all three learning categories, the Results tab, and the Playback & Annotation tab.

The screenshot shows the 'Admission' application interface. The left sidebar contains a 'DATASET' section with 'Upload Dataset' (Upload File, Open Project) and a 'CONFIGURATION' section with 'Dataset' (Data), 'Label' ((None)), and 'Features' (Serial_No, GRE_Score, TOEFL_Score, University_Rating, SOP, LOR, CGPA, Research, Admit_1). The main area has tabs for 'Data & Features', 'Models', 'Results', and 'Playback & Annotation'. The 'Data & Features' tab is active, showing 'Import', 'Feature Extraction', 'Analysis', and 'Visualization' sub-tabs. The 'Import' sub-tab is selected, displaying a 'DATASET PREVIEW' table with 8 columns: Serial_No, GRE_Score, TOEFL_Score, University_Rating, SOP, LOR, CGPA, Research, and Admit_1. A 'FILE SUMMARY' table on the right shows 500 rows, 9 columns, 0 missing cells, 0 duplicate rows, and 35.3 KB memory. A 'MISSING VALUES SUMMARY' table at the bottom shows 0 missing values for all columns.

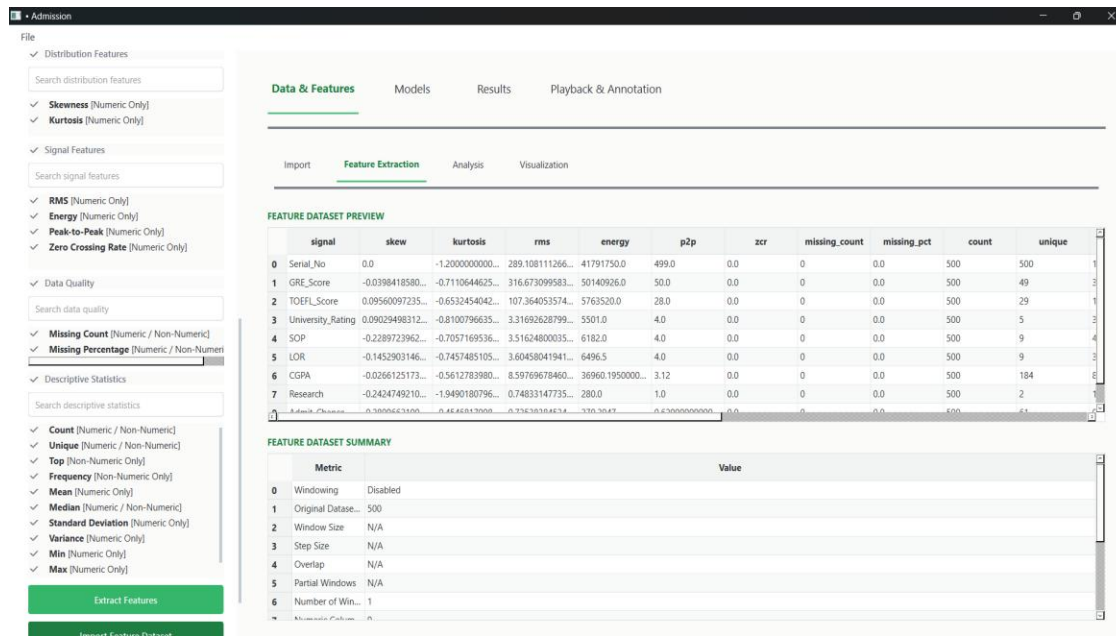
Serial_No	GRE_Score	TOEFL_Score	University_Rating	SOP	LOR	CGPA	Research	Admit_1
1	337	118	4	4.5	4.5	9.65	1	0.92
2	324	107	4	4.0	4.5	8.87	1	0.76
3	316	104	3	3.0	3.5	8.0	1	0.72
4	322	110	3	3.5	2.5	8.67	1	0.8
5	314	103	2	2.0	3.0	8.21	0	0.65
6	330	115	5	4.5	3.0	9.34	1	0.9
7	321	109	3	3.0	4.0	8.2	1	0.75
8	308	101	2	3.0	4.0	7.9	0	0.68

metric	value
0 Rows	500
1 Columns	9
2 Missing Cells	0
3 Duplicate Rows	0
4 Memory	35.3 KB

column	dtype	missing	missing_pct
1 GRE_Score	int64	0	0.00%
2 TOEFL_Score	int64	0	0.00%
3 University_Rating	int64	0	0.00%
4 SOP	float64	0	0.00%
5 LOR	float64	0	0.00%
6 CGPA	float64	0	0.00%

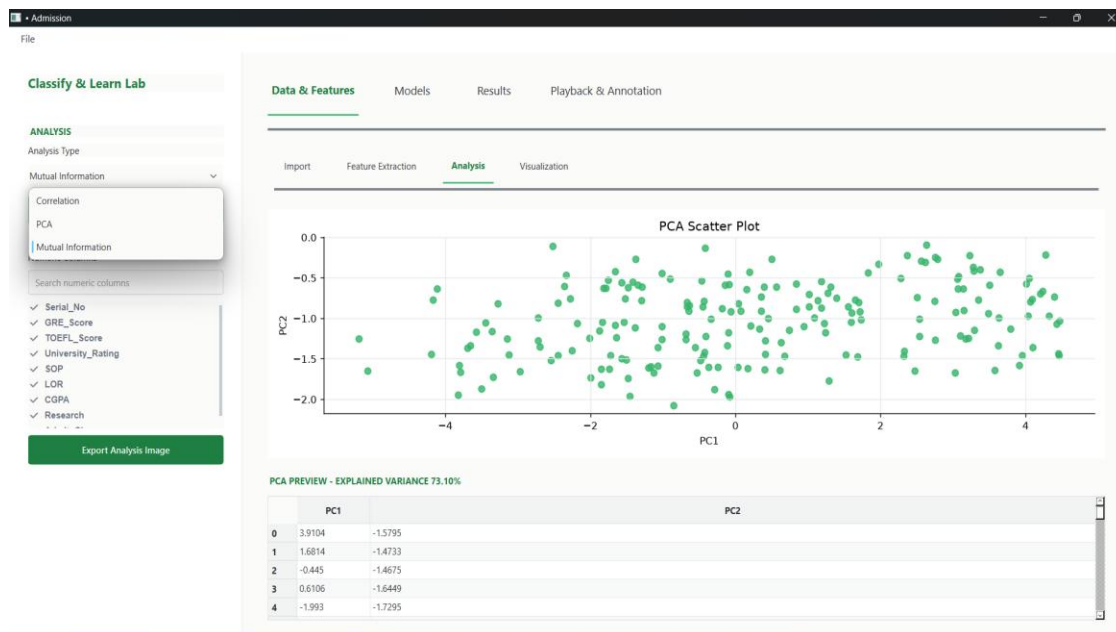
Import tab showing the dataset preview, file summary, and a per-column missing-values breakdown with data types and missing percentages.

This view confirms the dataset loaded cleanly with zero missing cells and zero duplicate rows and lets the user choose which columns to include as features before moving on to feature extraction or modeling.



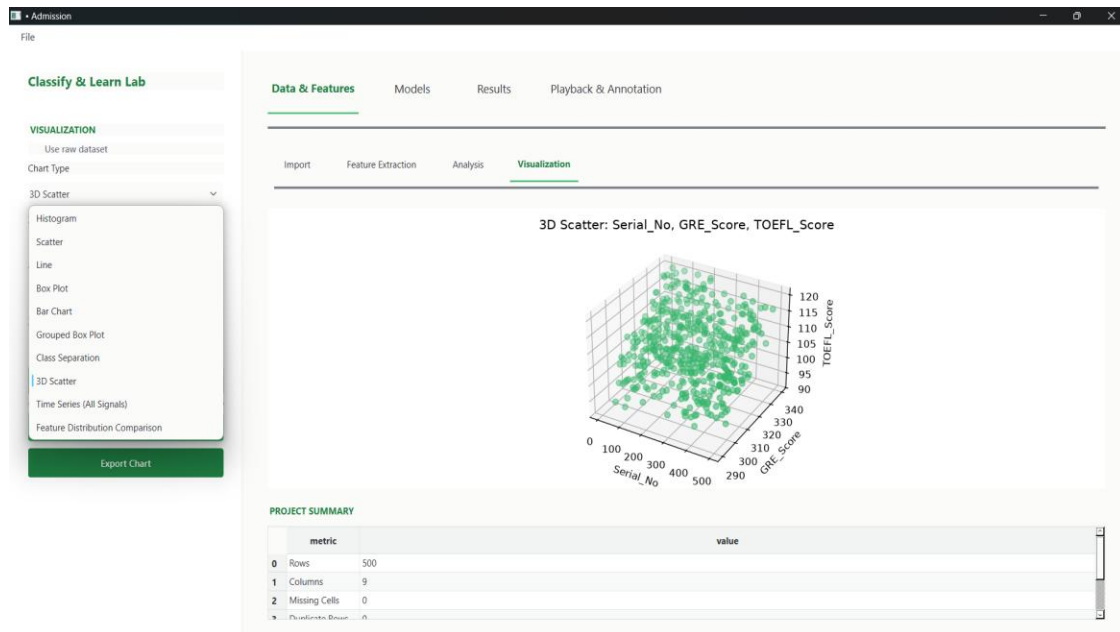
Feature Extraction tab, listing available statistical, distribution, and signal-based features (skewness, kurtosis, RMS, energy, peak-to-peak, zero-crossing rate) alongside the resulting feature dataset preview and summary.

Each row of the Feature Dataset Preview corresponds to one original column, and each column is a computed statistic for that column (e.g., skew, kurtosis, RMS) — turning the raw dataset into a feature-level summary that can be fed into feature-selection analysis or used directly for modeling.



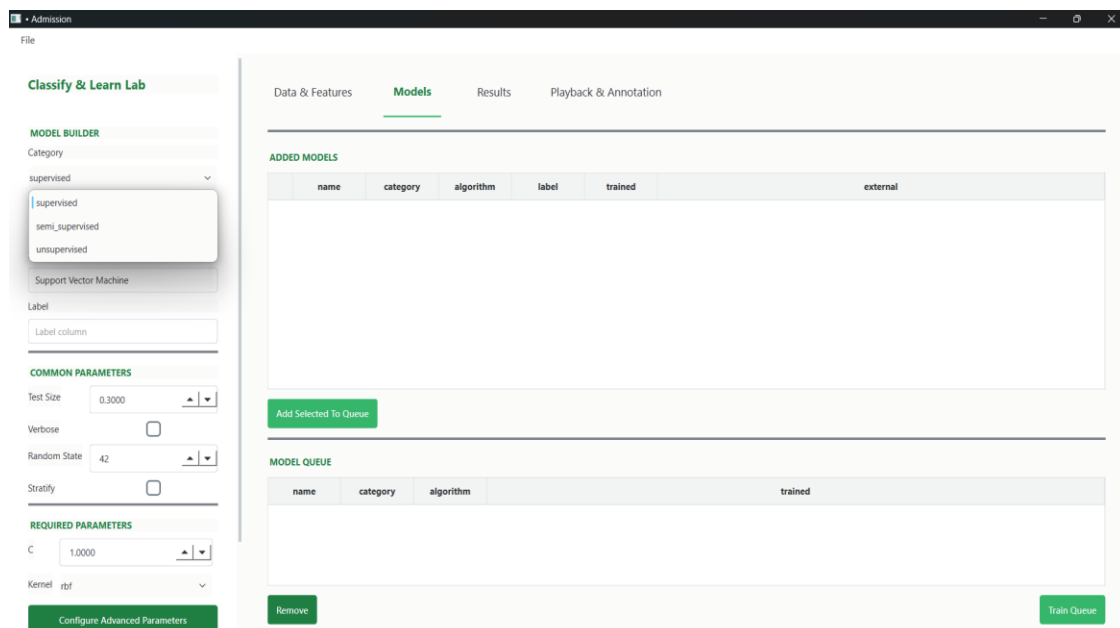
Analysis tab — PCA Scatter Plot projecting the dataset onto its first two principal components, with an explained-variance readout and a preview table of PC1/PC2 values per row.

The Analysis tab also offers Correlation and Mutual Information as alternative feature-relationship views (visible in the Analysis Type dropdown), giving the user several ways to understand which features carry the most information before selecting a modeling approach.



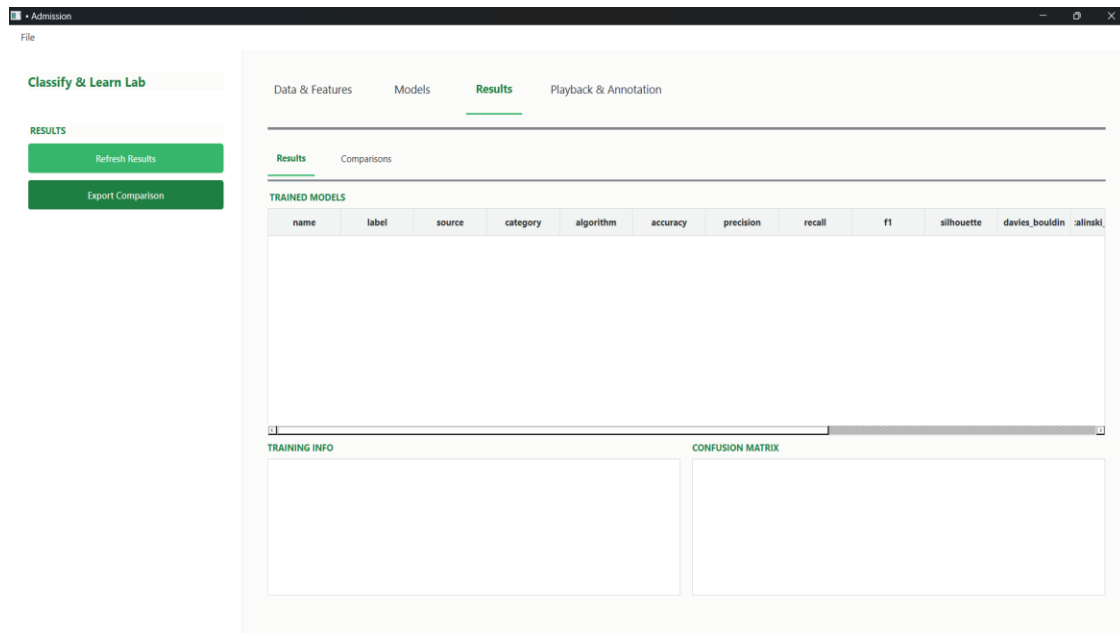
Visualization tab — we have the eleven chart types available in to the Chart Type dropdown.

The 3D Scatter view lets the user rotate and inspect relationships across three numeric columns at once, complementing the 2D chart types (Histogram, Scatter, Line, Box Plot, Bar Chart, Grouped Box Plot, Class Separation, Time Series, Feature Distribution Comparison, and Correlation Heatmap) shown earlier.



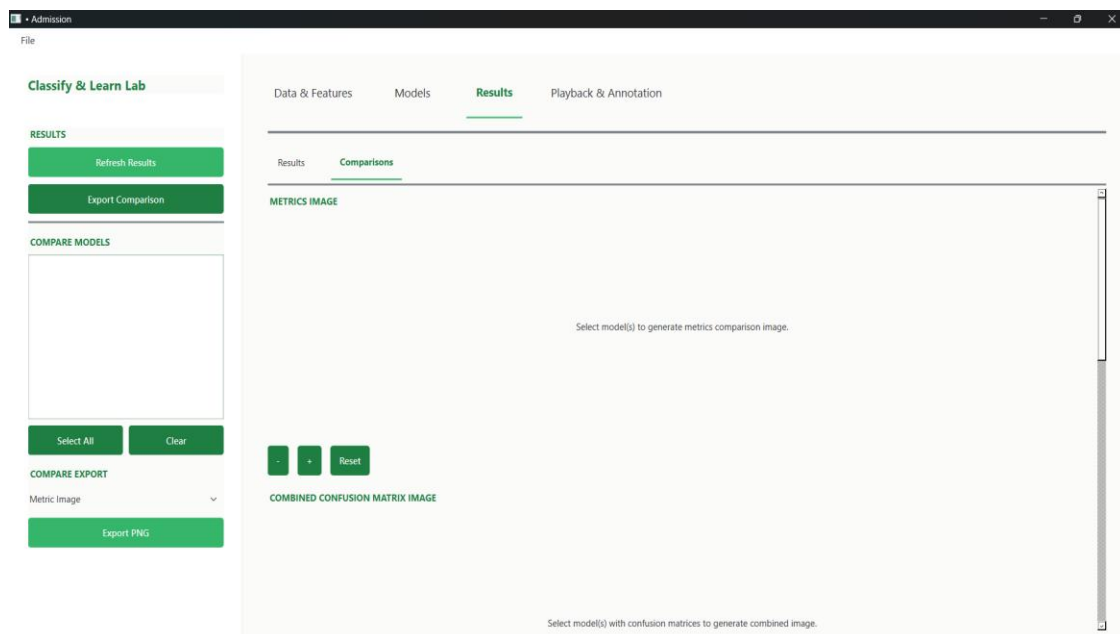
Models tab — Model Builder, showing the Category dropdown (supervised / semi supervised / unsupervised), an added Support Vector Machine model with its parameters, and the Add-to-Queue / Model Queue workflow for training multiple models in one batch.

This expanded Model Builder generalizes the original supervised-only training panel into a single interface that also covers semi-supervised and unsupervised learning and introduces a queue so several model configurations can be trained together rather than one at a time.

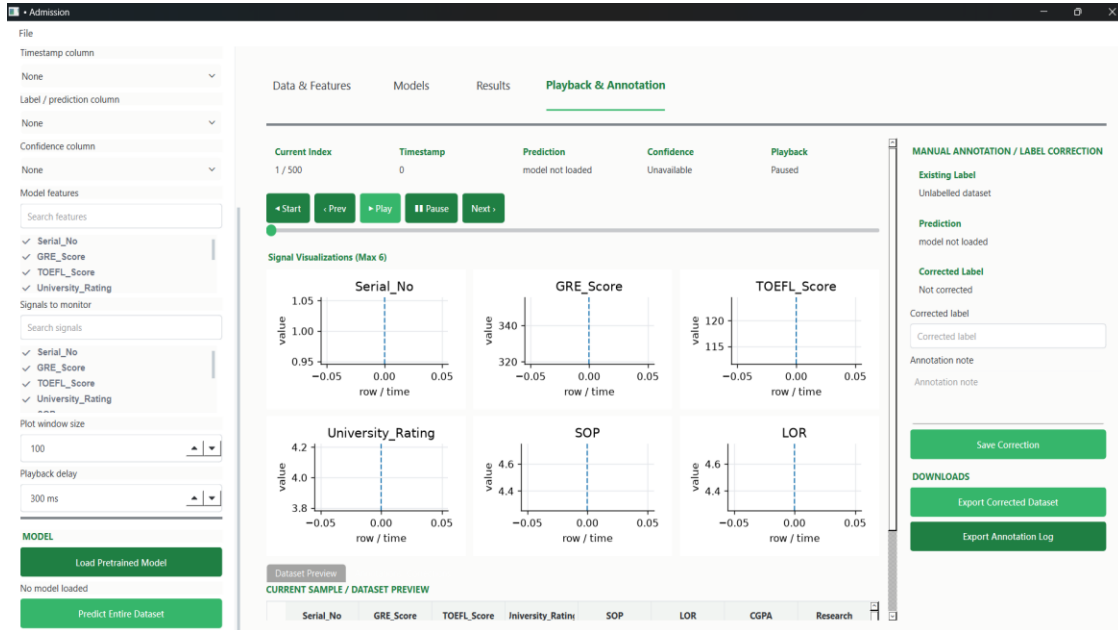


Results tab — Trained Models table listing name, label, source, category, algorithm, and both classification metrics (accuracy, precision, recall, F1) and clustering metrics (silhouette, Davies-Bouldin, Calinski-Harabasz) in one combined view, with Training Info and Confusion Matrix panels below.

By presenting classification and clustering metrics side by side, the Results tab gives a unified place to review any model trained through the Model Builder, regardless of which of the three learning categories it came from.



Comparisons tab, where selected models can be compared visually via a generated metrics image and a combined confusion-matrix image, with the result exportable as a PNG.



Playback & Annotation tab, showing synchronized per-signal plots (Serial_No, GRE_Score, TOEFL_Score, University_Rating, SOP, LOR) with playback controls, alongside a manual annotation/label-correction panel for reviewing and correcting predictions row by row.

This tab supports stepping or auto-playing through a dataset row by row while viewing each monitored signal, loading a pretrained model to generate predictions, and manually correcting labels with notes — with the corrected dataset and a full annotation log exportable afterward.

10. Conclusion

Classify & Learn Lab successfully delivers an end-to-end, GUI-driven machine learning workflow purpose-built for vehicle telemetry analysis — spanning data import and cleaning, feature engineering and selection, rich visualization, and model training/evaluation across supervised, semi-supervised, and unsupervised approaches — backed by a modular, independently testable Python codebase.

10.1 Future Work

- Wire the existing semi-supervised (run_ssl_workflow) and unsupervised (run_unsupervised_workflow) backends into the Models tab GUI, alongside the current supervised workflow.
- Add model editing and export directly from the Models tab (the backend functions already exist and are used by the CLI test scripts).
- Build out the Results tab to summarize trained-model comparisons and link back to the Playback & Annotation view.
- Expand automated GUI test coverage alongside the existing backend pytest suite.